

FITBUDDY

AI Fitness Plan Generator using Gemini Models

GROUP PROJECT DOCUMENTATION

Student / Group	GOMATHI. E
Course	BCA
Class	2nd BCA – A
Academic Year	2026–2027
Project Title	FitBuddy – AI Fitness Plan Generator using Gemini Models
Technologies	Python, FastAPI, Jinja2, HTML, CSS, Gemini AI, SQLite / SQLAlchemy

Group Members: Add all group member names and register numbers here.

TABLE OF CONTENTS

1. Project Overview
2. Brainstorming & Ideation
3. Requirement Analysis
4. Project Design
5. Project Planning
6. Project Development
7. Project Testing
8. Project Documentation
9. Project Demonstration
10. Source Code – main.py
11. Source Code – result.html
12. Conclusion

1. Project Overview

FitBuddy is a web-based AI fitness planning application designed to generate personalized workout guidance from user-provided information. The system accepts details such as name, age, weight, fitness goal and workout intensity, then uses a Gemini model to produce a structured plan. The project also supports feedback-based plan refinement and an administrative view of stored user information.

The application uses FastAPI for backend routing and request handling, Jinja2 for dynamic HTML pages, and SQLite/SQLAlchemy for storing application data. The reference project describes pages such as the home page, personalized result page, feedback page and view-all-users page.

Main Objectives

- Generate a personalized 7-day workout plan.
- Use AI to tailor the output to the user's fitness goal and selected intensity.
- Provide simple nutrition or recovery guidance.
- Allow users to submit feedback for plan improvement.
- Store and display relevant user and plan information.

2. Brainstorming & Ideation

The idea for FitBuddy was developed around the need for a simple fitness assistant that can generate personalized guidance rather than giving the same plan to every user. The team identified AI-based personalization, an easy web interface, feedback, and stored user information as useful features.

3. Requirement Analysis

The functional requirements include collecting user details, generating a workout plan, displaying the generated result, accepting feedback, updating the plan where supported, and providing an administrative/user-list view. The non-functional requirements include usability, reasonable response time, maintainable code, and protection of sensitive configuration such as the Gemini API key.

4. Project Design

The system follows a simple web architecture. The browser sends form data to the FastAPI backend. The backend validates and processes the input, communicates with the Gemini service, stores relevant data through the database layer, and renders a Jinja2 template containing the result.

5. Project Planning

The project can be organized into milestones: selecting the AI model and architecture, developing backend functionality and routes, creating the frontend templates, connecting the database, testing the complete flow, and preparing the application for local demonstration. The reference project similarly organizes work into model/architecture, core development, route development, frontend development and deployment milestones.

6. Project Development

During development, the FastAPI application handles routes and user input. Jinja2 templates provide the web pages. The reference structure identifies index.html for collecting input, result.html for displaying the generated AI plan, and all_users.html for the administrative view. Environment variables are used for the Gemini API key rather than placing the key directly in source code.

7. Project Testing

Testing is performed by checking the main user flow from entering details to receiving a generated result. Form validation, successful AI response, result-page rendering, feedback submission, database storage, and the all-users view should be checked before demonstration.

8. Project Documentation

The documentation explains the purpose of FitBuddy, system requirements, architecture, development process, testing approach, user interface pages, source-code highlights and final result.

9. Project Demonstration

For the demonstration, the team can show the home page, enter sample user details, generate a personalized plan, display the result, demonstrate feedback if implemented, and show the stored-user/admin page if available. The reference project describes the home page and personalized

workout result as important parts of the demonstration.

10. Source Code – main.py

```
import os
from fastapi import FastAPI, Request, Form
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse

app = FastAPI(title="FitBuddy - AI Fitness Plan Generator")
templates = Jinja2Templates(directory="templates")

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        "index.html", {"request": request}
    )

@app.post("/generate", response_class=HTMLResponse)
async def generate(
    request: Request,
    name: str = Form(...),
    age: int = Form(...),
    weight: float = Form(...),
    goal: str = Form(...),
    intensity: str = Form(...)
):
    # Connect this section to the Gemini service in your project.
    workout_plan = "AI-generated personalized workout plan"
    return templates.TemplateResponse(
        "result.html",
        {
            "request": request,
            "name": name,
            "age": age,
            "weight": weight,
            "goal": goal,
            "intensity": intensity,
            "workout_plan": workout_plan
        }
    )
```

Note: This is a documentation-friendly code extract. Keep your group's actual working main.py code in the final project folder.

11. Source Code – result.html

```
<!DOCTYPE html>
<html>
<head>
    <title>FitBuddy Result</title>
</head>
<body>
    <h1>Your Personalized Fitness Plan</h1>

    <p><b>Name:</b> {{ name }}</p>
    <p><b>Age:</b> {{ age }}</p>
    <p><b>Weight:</b> {{ weight }} kg</p>
    <p><b>Goal:</b> {{ goal }}</p>
    <p><b>Intensity:</b> {{ intensity }}</p>

    <h2>Workout Plan</h2>
    <div>
        {{ workout_plan }}
    </div>
</body>
</html>
```

Note: Replace this extract with the exact result.html code used by your group if your file has additional styling or fields.

12. Conclusion

FitBuddy demonstrates how generative AI can be combined with a web application to provide personalized fitness planning. The application collects user information and presents a structured workout plan based on the selected fitness goal and intensity. Its FastAPI backend, Jinja2 templates and database layer provide the basic structure needed for a complete web solution.

A key part of the project is personalization: the plan is intended to respond to the user's provided information rather than displaying one fixed plan. The project also provides a foundation for feedback-based refinement and administrative monitoring of stored user data, as described in the reference project.

Overall, the project gives the group practical experience in FastAPI, HTML/Jinja2 templates, database handling and integration with a generative AI service. Before final submission, the group should add all member names and register numbers on the cover page.